Testing Out Llama Cpp Grammar Constraint-based Sampling

A promising new sampling method

Paolo Rechia · Follow
Published in Better Programming
5 min read · Aug 18, 2023

Listen Share More

Photo by Element5 Digital on Unsplash

Some weeks ago I heard of an innovative work going in the `llama-cpp` repository that would add grammar constraint-based sampling to the repository.

What does that mean? In summary, this new feature would give us a huge potential for generating complex structural output, such as JSON and API calls without the needing of fine-tuning the model.

But additionally, it also means in theory we can generate syntax-error-free code! All we need to do is feed in the grammar of the target programming language, and we'll have a code that "compiles", because the generated output will necessarily follow the grammar.

I personally expect this feature to be so powerful that it could replace libraries like Guidance altogether.

Of course, there's an additional complexity involved, which is writing context-free grammar. If you want to understand better how this is implemented, check the original PR.

In this article, we'll focus on building a full working example in Python that leverage the power of this new feature.

Our agenda for today:

1. Choosing a model
2. Preparing and testing our environment
3. Writing a simple grammar to understand how it works

The example code snippets are available here.

Choosing a model

First things first — a lot has happened in the last few months, so we should choose an up-to-date model according to some benchmarks.

Fortunately, I've found this very interesting leaderboard which ranks different Large Language Models according to different benchmarks.

A model that caught my attention is this `llama2-13b-megacode2_min100`:

Click on the links for more details!

Rank	Size	Model	Total	MT-Bench	CoT	HumanEval+	LM-Eval
1		GPT-4-0613		8.89	0.84	0.80	
2		GPT-3.5-Turbo-0613		8.22	0.64	0.67	
3		GPT-3.5-Turbo-0301		8.11	0.65	0.67	
4	70B	WizardLM 70B V1.0	66.96	7.68	0.63	0.40	72.23
5	70B	Stable Beluga 2		7.42	0.58	0.36	
6	70B	LLaMA-2 70B Chat	61.16	7.10	0.47	0.29	71.54
7	13B	andreaskoepf/llama2-13b-megacode2_min100	58.78	6.54	0.41	0.29	71.29
8	70B	OpenAssistant/oasst-llama2-70b-1		6.85		0.29	
9	40B	OpenAssistant/falcon-40b-megacode2-oasst		6.53	0.43	0.24	
10	13B	andreaskoepf/llama2-13b-megacode2-oasst	58.65	6.56	0.38	0.29	71.91
11	33B	Open-Assistant Llama-33B SFT-7e3	57.76	6.63	0.36	0.25	71.94

Top 11 models in Leaderboard

Its performance is comparable to much larger models, although still far behind the open-source leader `WizardLM 70B`. Nonetheless, I want to test something that is of a reasonable size to test. So this will do.

Unfortunately, I couldn't find a GGML version of it, so I took instead the `oasst` version that scores slightly lower (to be honest, almost the same). Here's the model converted by The Bloke. I downloaded the `q4_K_M` version binary file.

Preparing our environment

I've initially checked llama-cpp-python and found out that we also have Python bindings available since last week! But unfortunately, I've tested it, and did not work as expected... yet! So I created a bug report — in the meantime, I was able to get results using `llama-cpp` server directly. My code is mostly based on this Pull Request example so the credits go to the original author.

First, install your `llama.cpp` from source following the instructions here. Once you are able to build, you can access `build/bin` and find the `server` binary there.

Save your downloaded model next to this server, and then start it with:

```
./server -m llama2-13b-megacode2-oasst.ggmlv3.q4_K_M.bin
```

In a separate file, add the following code:

```
"""Code based on https://github.com/ggerganov/llama.cpp/pull/2532"""
import requests

grammar = r'''
root ::= answer
answer ::= "Good"
'''

prompts = [
    "How's the weather in London?",
]

for prompt in prompts:
    data_json = { "prompt": prompt, "temperature": 0.1, "n_predict": 512, "stream": False, "grammar": grammar }

    resp = requests.post(
        url="http://127.0.0.1:8080/completion",
        headers={"Content-Type": "application/json"},
        json=data_json,
    )
    result = resp.json()["content"]

    print(f"Prompt: {prompt}")
    print(f"Result: {result}\n")
```

Install the `requests` with `pip install requests` and run this script. If all goes well, you should see the following output:

```
(env-llama-grammar) (base) paolo@paolo-MS-7D08:~/llama-grammar-example$ python minimal_example.py
Prompt: How's the weather in London?
Result: Good
```

Your environment works! Let's break this down.

Writing a simple grammar

In the previous example, we're constraining the token generation using this grammar:

```
grammar = r'''
root ::= answer
answer ::= "Good"
'''
```

It always outputs `Good`. This example is not so exciting! Let's try some more interesting examples. Let's extend the grammar to resolve between three options, and test it with three different prompts. We'll do this using the `or` logical operator which is represented by the pipe (`|`) character.

```
grammar = r'''
root ::= answer
answer ::= ("Sunny." | "Cloudy." | "Rainy.")
'''

prompts = [
    "How's the weather in London?",
    "How's the weather in Munich?",
    "How's the weather in Barcelona?",
]
```

It seems like this model likes Cloudy weather, it returns this answer most of the time. Did get a sunny once, so we know it works as expected:

```
(env-llama-grammar) (base) paolo@paolo-MS-7D08:~/llama-grammar-example$ python3 weather_example.py
Prompt: How's the weather in London?
Result: Sunny.

Prompt: How's the weather in Munich?
Result: Cloudy.

Prompt: How's the weather in Barcelona?
Result: Cloudy.
```

If you've used frameworks like Guidance before, which I wrote extensively about in the past, we've just replicated their `Select` functionality!

But it can be more interesting than that. We can nest the token structure in the grammar. For instance, let's make a model that replies about the weather or complains:

```
grammar = r'''
root ::= answer
answer ::= (weather | complaint | yesno)
weather ::= ("Sunny." | "Cloudy." | "Rainy.")
complaint ::= "I don't like talking about the weather."
yesno ::= ("Yes." | "No.")
'''
```

In summary, this grammar describes an answer that can be either about the weather, a complaint, or just a yes/no.

Let's see it in action:

```
(env-llama-grammar) (base) paolo@paolo-MS-7D08:~/llama-grammar-example$ python3 complaining_example.py
Prompt: <|im_start|You are a weather predictor. Answer with either Sunny, Cloudy or Rainy. How's the weather in
Result: Sunny.

Prompt: <|im_start|How's the weather in Munich? Is it sunny? Answer with yes or no.<|im_end|><|im_start|>
Result: Yes.

Prompt: <|im_start|How's the weather in Barcelona? Try to complain about this.<|im_end|><|im_start|>
Result: I don't like talking about the weather.
```

Worked pretty! But not before I tweaked the prompt a little bit. You may notice some special tokens there, that flag when the user input starts and ends, as well as when the assistant reply starts.

Before adding the special tokens, the model would just complain all the time — so keep in mind that even using grammar still requires a good model and some prompting to get the most out of it.

Conclusion

Of course, studying the basics about compilers does help in understanding this type of grammar, but you don't need to get too complex into this for it to be useful. Also, you can probably find useful examples out there. One of the examples provided in `llama-cpp` repository implements a JSON grammar for instance, which is super useful if you're making the model call a REST API.

One thing I might try next is pulling the grammar of Python and generating syntax-error-free code.

Hope you found this new feature as exciting as I did! Cheers!

Large Language Models | Python | Llama 2 | Machine Learning | AI

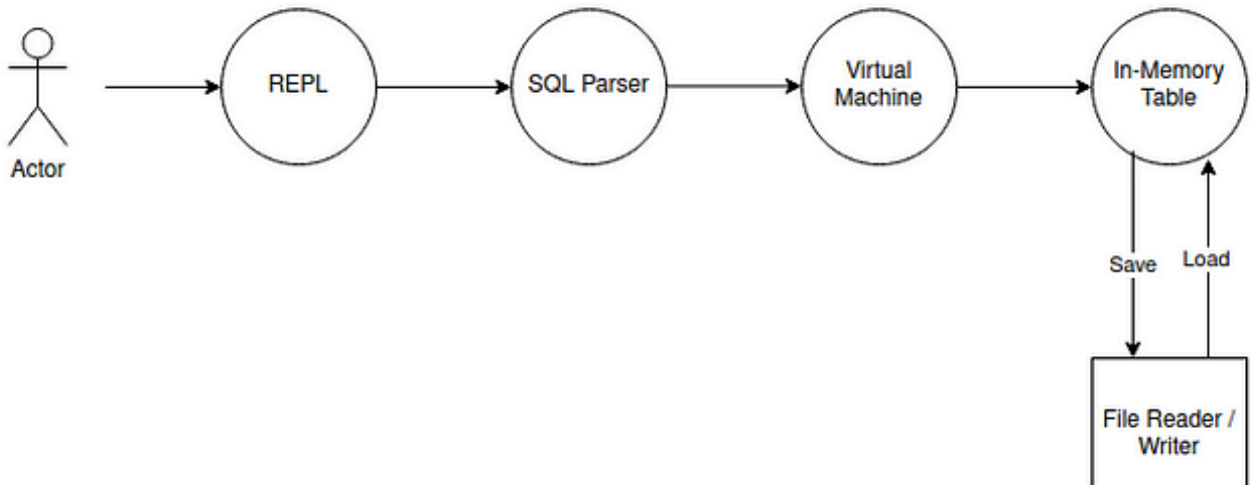
Follow

Written by Paolo Rechia

598 Followers · Writer for Better Programming

Software Developer / Data Engineer - Connect with me on Linkedin: https://www.linkedin.com/in/paolo-rechia/

More from Paolo Rechia and Better Programming



Paolo Rechia

Building a Database From Scratch in Rust—Part 1

Not as hard as it sounds

Dec 31, 2023 360 6

Benoit Ruiz in Better Programming

Advice From a Software Engineer With 8 Years of Experience

Practical tips for those who want to advance in their careers

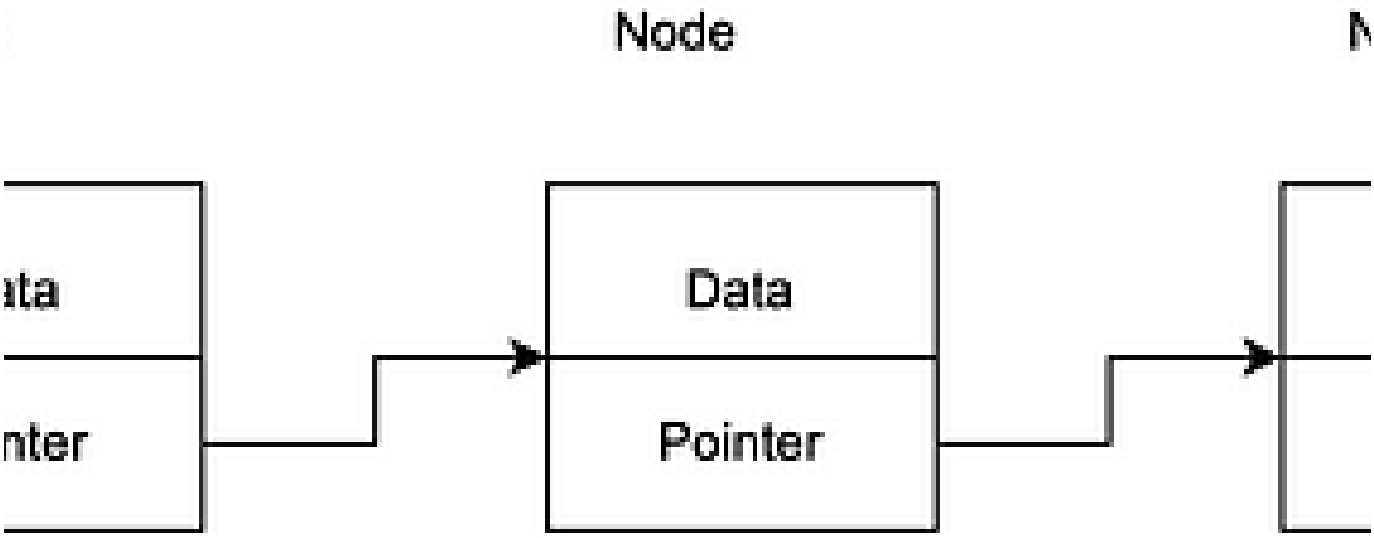
Mar 21, 2023 17K 296

Emily Dresner in Better Programming

Why an Engineering Manager Should Not Review Code

When discussing team organization, I am often asked: “Why don’t you have the tech lead manage the team?” My response is to hiss like a...

May 11, 2023 4.5K 49



Paolo Rechia in Better Programming

Learn Rust by Building a Linked List

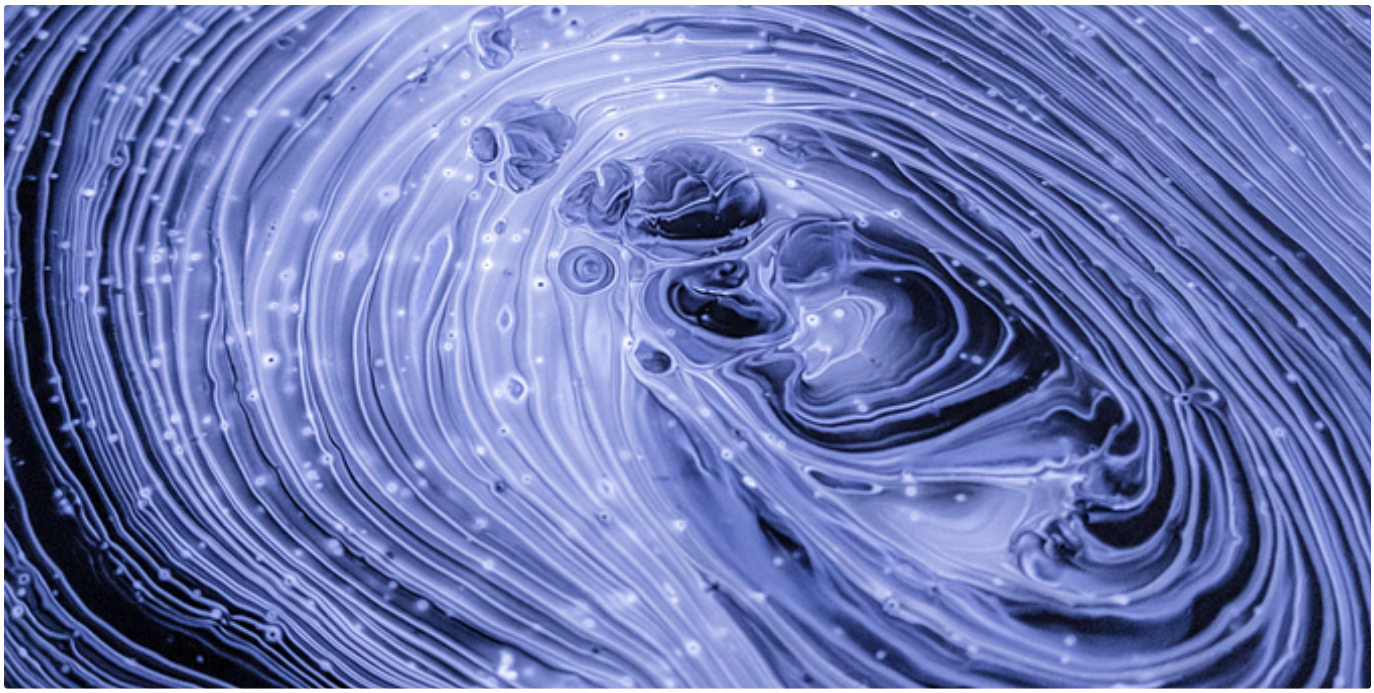
Implementing using the unsafe and safe way

Apr 10, 2023 209 5

See all from Paolo Rechia

See all from Better Programming

Recommended from Medium



Dmitrii Eliuseev in Towards Data Science

16, 8, and 4-bit Floating Point Formats—How Does it Work?

Let's go into bits and bytes

Sep 30, 2023 480 4

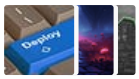
Louis Chan in Towards Data Science

You Don't Need an LLM Agent

The Why & The Alternative

3d ago 112 5

Lists

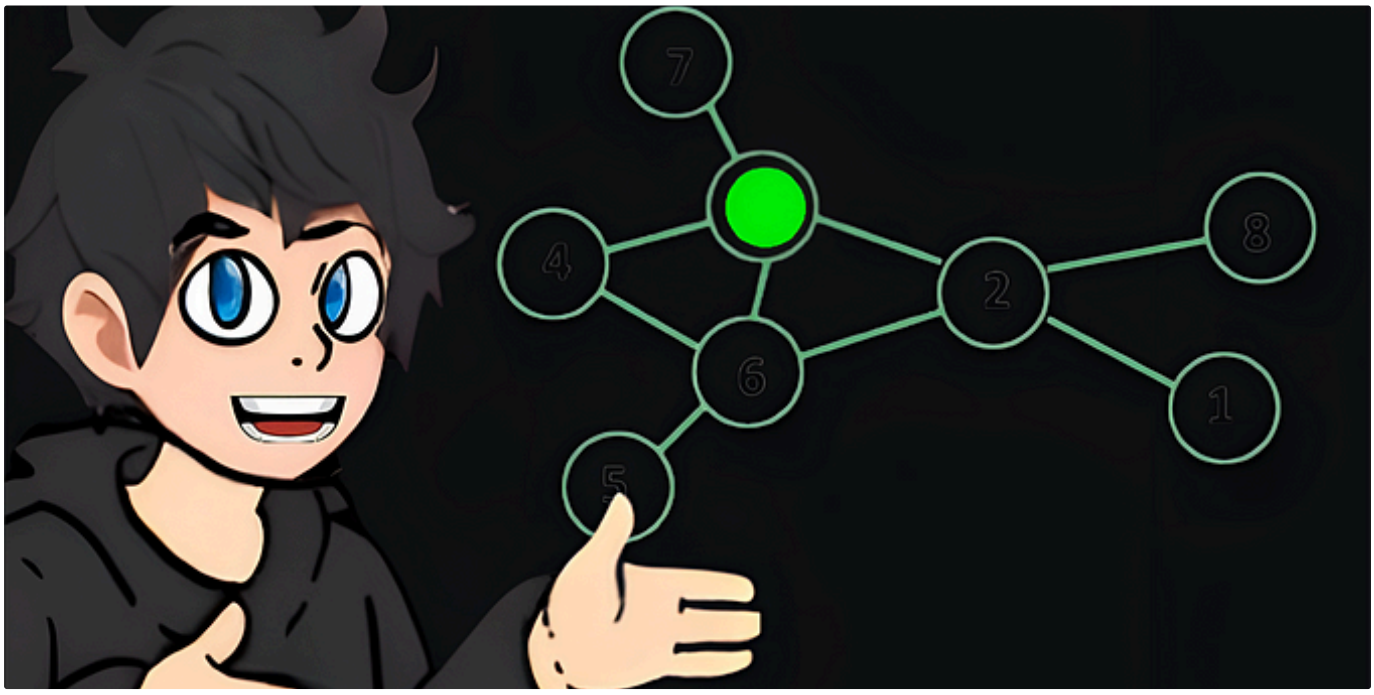


Predictive Modeling w/ Python
20 stories · 1374 saves

Practical Guides to Machine Learning
10 stories · 1665 saves

Natural Language Processing
1583 stories · 1128 saves

Coding & Development
11 stories · 699 saves

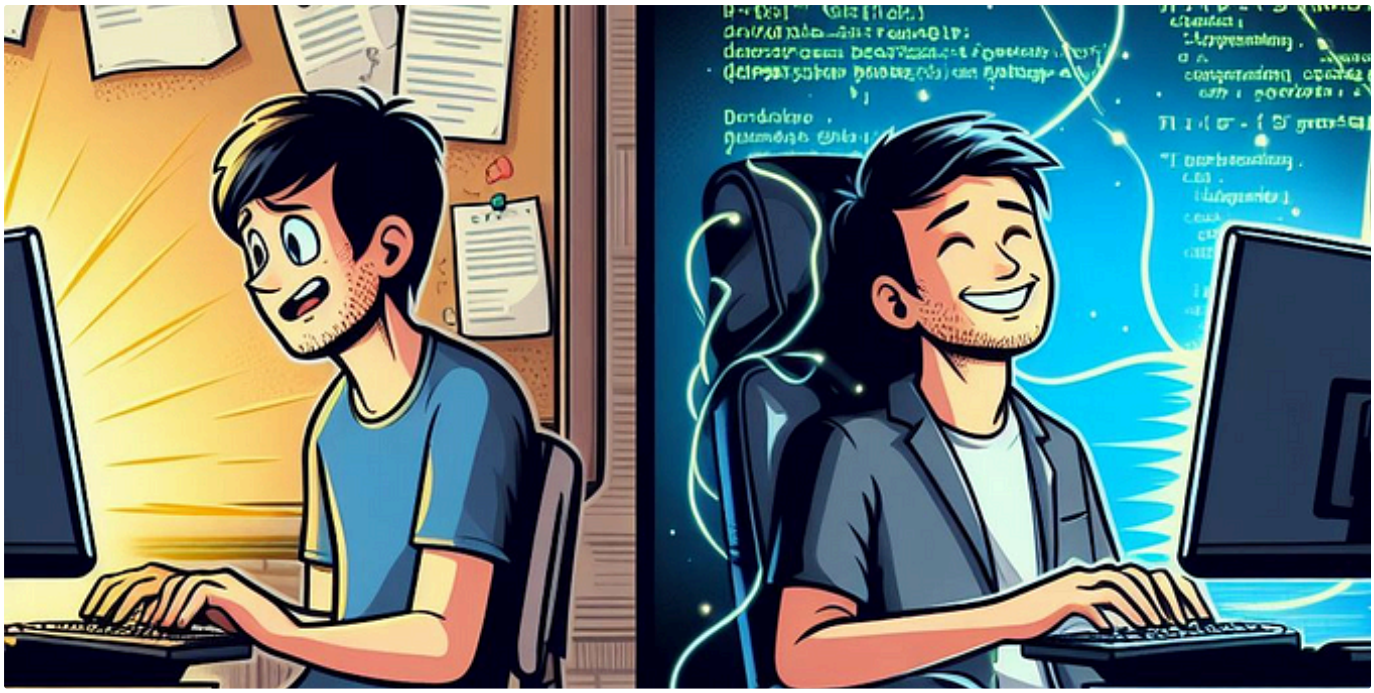


Gao Dalie (高達烈) in Towards AI

Langchain + Graph RAG + GPT-4o Python Project: Easy AI/Chat for your Website

This is Graph and I have a super quick tutorial showing how to create a fully local chatbot with Langchain, Graph RAG and GPT-4o to make a...

Jul 7 · 788 · 2

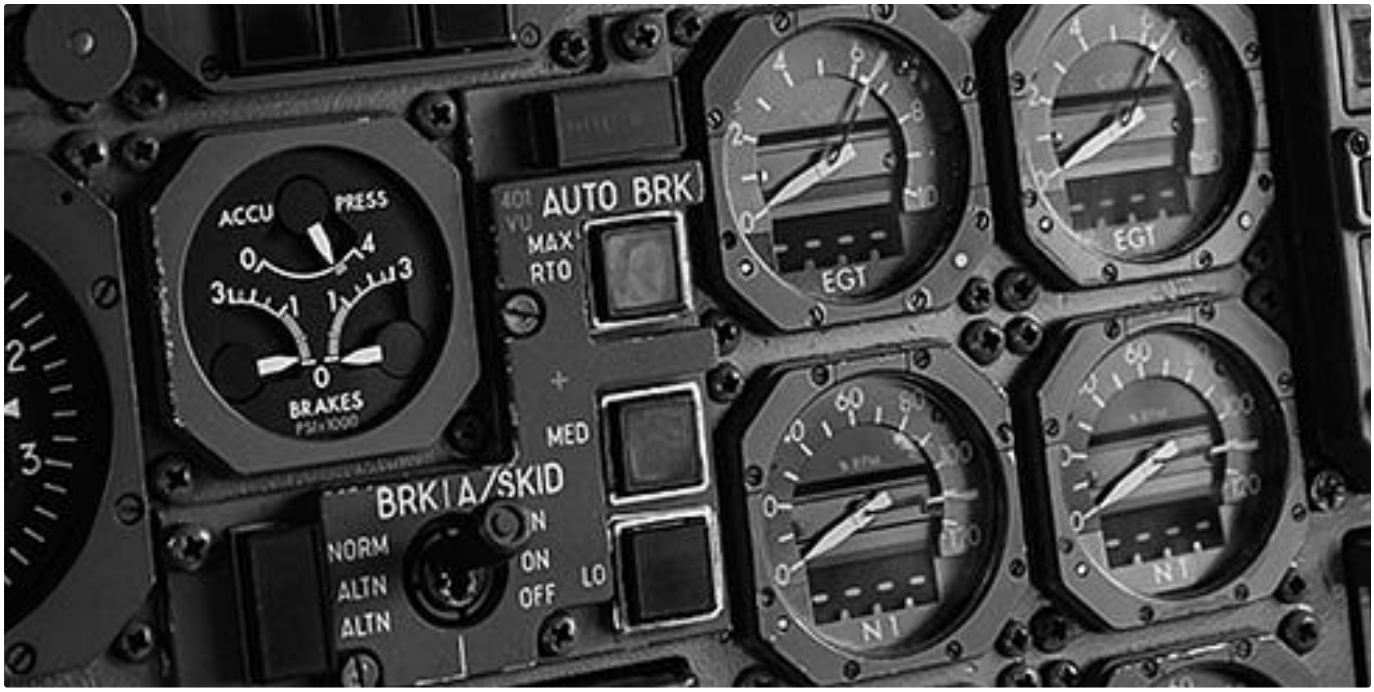


Abhay Parashar in The Pythoneers

17 Mindblowing Python Automation Scripts I Use Everyday

Scripts That Increased My Productivity and Performance

5d ago 2.7K 20



Matt Chinnock

Controlling Large Language Model Output with Pydantic

Hint: It doesn't actually involve using any dials.

Mar 7 136 2